

Rapport de veille technique

REST vs gRPC pour une plateforme IoT industrielle

Contexte. Ce document synthetise la veille et le benchmark conduits pour SignalWatch, startup IoT en phase de levee de fonds. L'objectif est d'appuyer le choix d'un style d'API inter-services (REST/JSON sur HTTP vs gRPC/Protobuf sur HTTP/2) pour un domaine a forte cadence d'evenements (ordre de 10 000 evenements par minute) et des contraintes de latence et d'exploitation.

Perimetre et methodologie

Deux microservices Rust equivalents ont ete implements: un service REST (Axum) et un service gRPC (Tonic/Prost), avec le meme modele metier (capteurs) et des regles d'erreur alignees. Les charges sont produites par k6 (REST) et ghz (gRPC) selon trois scenarios: A lecture unitaire, B ecriture, C rampe de concurrence. Les resultats bruts JSON sont archives dans benchmark/results/ et un rapport consolide est genere (make report) vers benchmark/results/report-latest.md.

Resultats (instantane de reference)

Un jeu de resultats de reference a ete produit le 12 mai 2026 (timestamp BENCH_TIMESTAMP = 20260512-signalwatch-final) sur machine de developpement locale. Chiffres indicatifs: sur le scenario A (lecture), le service REST a montre une latence mediane d'environ 0,14 ms et un debit d'environ 43,7 k req/s; le service gRPC autour de 0,46 ms en p50 et 16,3 k req/s en debit rapporte par ghz. Sur le scenario B (ecriture), REST ~0,13 ms (p50) et ~24,3 k req/s; gRPC ~0,22 ms (p50) et ~15,6 k req/s. La rampe C fait monter la latence avec la concurrence (exemple observe: gRPC p50 jusqu'a ~4,8 ms pour la derniere etape a 100 connexions simultanees). Ces valeurs dependent du materiel et ne constituent pas une garantie de performance absolue; elles servent de ligne de base reproductible dans ce depot.

Analyse et eco-conception

REST facilite le debug et l'integration avec les outils Web standards; les payloads JSON sont plus volumineux que Protobuf. gRPC offre un contrat fort (proto), du multiplexage HTTP/2 et des messages compacts, au prix d'une courbe d'outillage (protoc, generation de code). Pour l'empreinte runtime, comparer CPU/RAM sous charge reelle et la taille moyenne des reponses reste indispensable avant arbitrage definitif.

Conclusion

Le depot fournit une base technique homogene pour comparer REST et gRPC sur un cas IoT representatif. La decision doit combiner ces mesures avec les contraintes produit (equipes, observabilite, compatibilite gateway, mobile), la gouvernance du schema (proto) et les exigences de compatibilite ascendante.